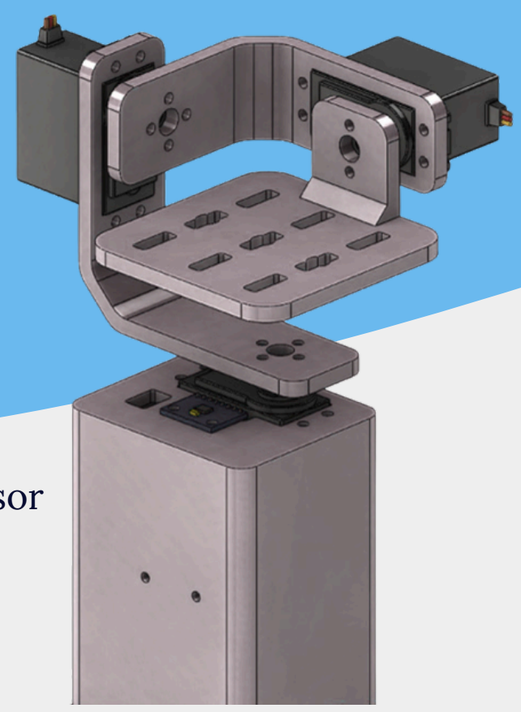
Self-Leveling Stabilizer

Akshat, Akshat Garg, Aryan Chaturvedi, Ayush Sharma,Pathak Prince, Rahul Das

Department of Materials Science and Engineering
Indian Institute of Technology Kanpur

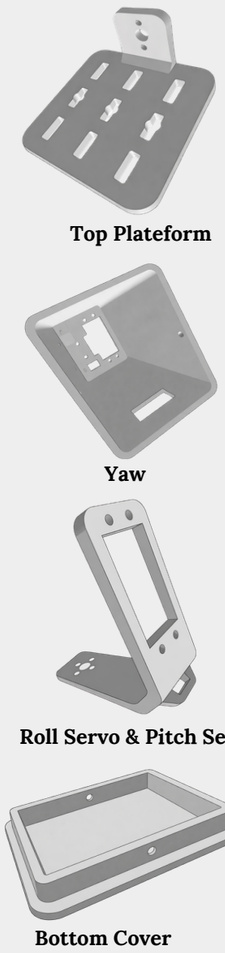


ABSTRACT

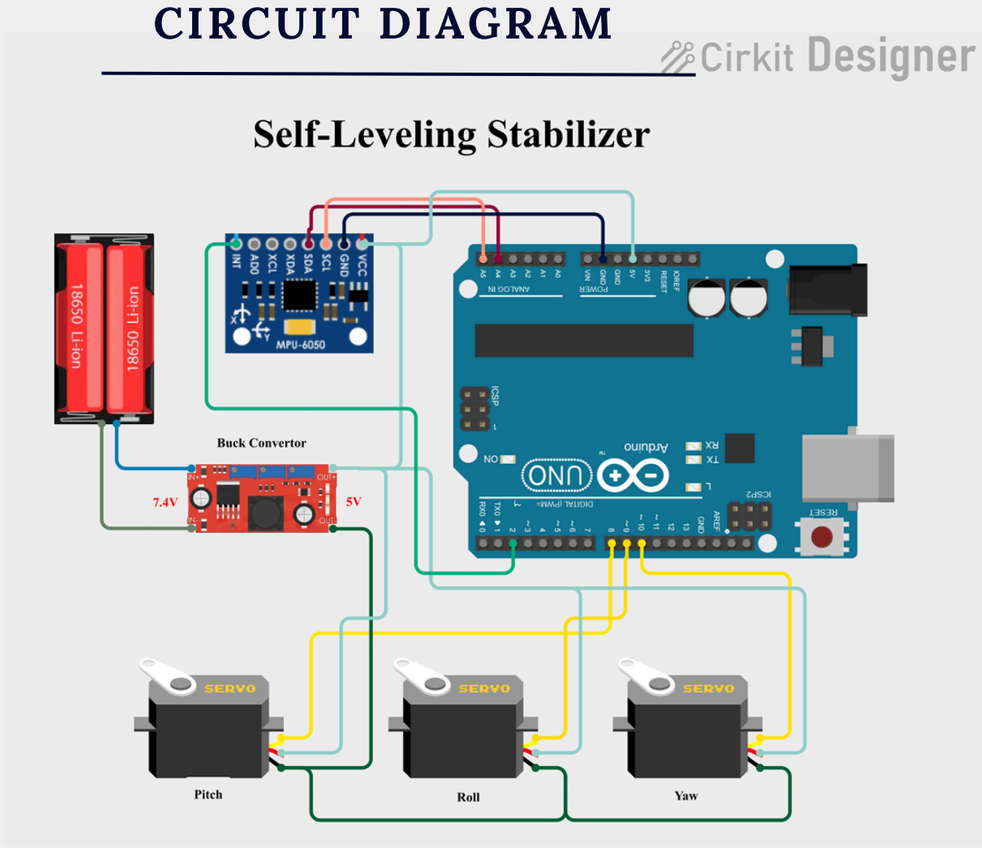
This project presents a self-leveling stabilizer using an Arduino and MPU6050 IMU. Complementary filter-based sensor fusion and closed-loop control enable real-time tilt correction using servo motors. The system maintains horizontal alignment under disturbances, with minor limitations due to calibration sensitivity and Euler-angle coupling.

SYSTEM ARCHITECTURE & COMPONENTS

Component	Specification	Function
Arduino Uno	ATmega328P MCU	Central controller for data processing
MPU6050 IMU	3-axis accelerometer + gyroscope	Measures orientation (acceleration + angular velocity)
Servo Motor (MG996R)	High-torque PWM servo	Actuation for tilt correction
Li-ion Battery (18650)	3.7V rechargeable cells	Portable power supply
Buck Converter	DC-DC step-down	Voltage regulation for stable operation
Mechanical Frame	PLA-based 3D printed structure	Structural support and platform stabilization
Jumper Wires	Standard connectors	Electrical interfacing



CIRCUIT DIAGRAM



3D FABRICATION & DESIGN

The structure was designed in CAD and fabricated using FDM-based 3D printing (PLA). The modular design ensures low inertia and efficient multi-axis stabilization.

SIMULATION & VISUALIZATION

Real-time orientation data was visualized using Processing (Java-based GUI) via serial communication, enabling 3D rendering of yaw, pitch, and roll motion.

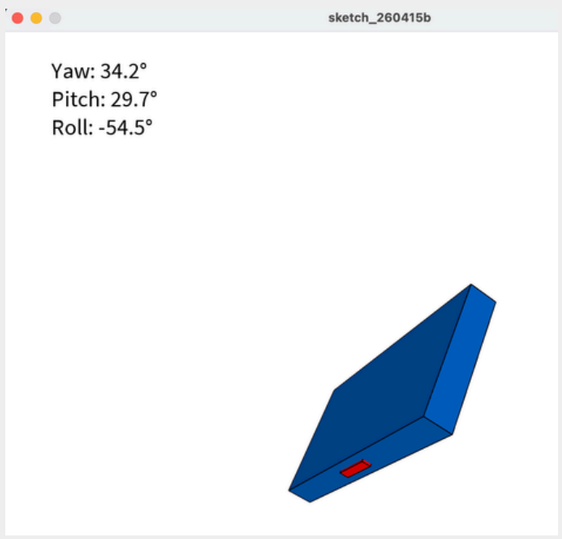
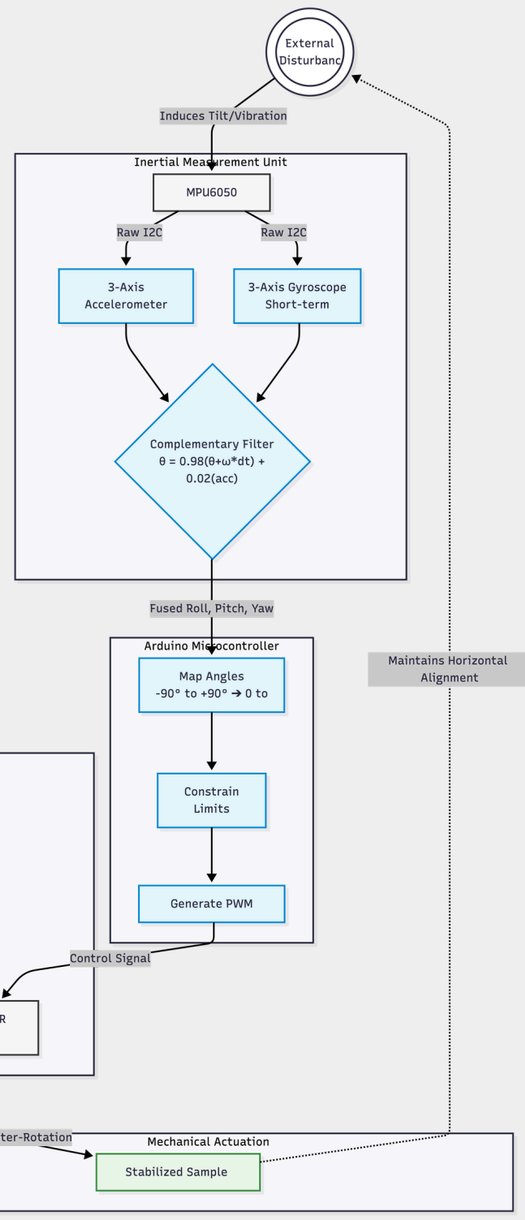


FIGURE: REAL-TIME 3D ORIENTATION VISUALIZATION (YAW, PITCH, ROLL) USING PROCESSING VIA SERIAL IMU DATA.

CONTROL IMPLEMENTATION

IMU data is acquired and processed using a complementary filter to estimate orientation. The controller computes tilt error and maps it to servo actuation signals, enabling closed-loop stabilization through continuous feedback.



Orientation Estimation

```
mpu.update();

float pitch = -mpu.getAngleY();
float roll  =  mpu.getAngleX();
float yaw   = -mpu.getAngleZ();
```

Control Mapping

```
int pitchAngle = map(pitch, -90, 90, 0, 180);
int rollAngle = map(roll, -90, 90, 0, 180);
int yawAngle = map(yaw, -90, 90, 0, 180);

pitchAngle = constrain(pitchAngle, 0, 180);
rollAngle = constrain(rollAngle, 0, 180);
yawAngle = constrain(yawAngle, 0, 180);
```

Servo Actuation

```
pitchServo.write(pitchAngle);
rollServo.write(rollAngle);
yawServo.write(yawAngle);
```

WORKING PRINCIPLE

The system operates on a feedback mechanism where IMU-based orientation is continuously estimated using sensor fusion. The controller computes tilt error and actuates servos in the opposite direction to maintain horizontal alignment.

CONCLUSIONS

Real-time stabilization was achieved using complementary filter-based sensor fusion and closed-loop control, enabling effective tilt correction under disturbances. Calibration sensitivity and Euler-angle-based axis coupling introduced minor limitations, which can be improved using quaternion-based orientation estimation.

REFERENCES

- Motion Control, element14 Community
- MPU6050 3-Axis , Instructables
- MPU-6050 Product Specification, InvenSense
- Arduino-Based 2-Axis Stabilizer
- MPU6050 Gyroscope and Accelerometer Tutorial

ACKNOWLEDGEMENTS

We thank Prof. Sandeep Sangal and Prof. Somnath Bhowmick for guidance, Gyan Prakash Bajpai along with lab staff for technical assistance, and Rakesh Dixit, Anil Verma, and I.P. Singh for 3D printing support.